What Must Kubernetes-Based Plant Supervision Demonstrate? Lessons from a First Implementation

Matheus Corteletti Delfino
Control and Automation Engineering
Instituto Federal do Espírito Santo (Ifes)
Serra, ES, Brazil
matheus.corteletti.del@gmail.com

Rafael Emerick Zape de Oliveira
Control and Automation Engineering
Instituto Federal do Espírito Santo (Ifes)
Serra, ES, Brazil
rafael.emerick@ifes.edu.br

***Abstract*—Kubernetes keeps software systems close to a declared desired state, and recent work uses it to run industrial control software. This work in progress asks a different question: what must an implementation demonstrate before one can say that a process plant is under Kubernetes-based supervision? We report our experience with this question. We built a simulation of the Tennessee Eastman Process Benchmark that publishes its signals over OPC UA, a historian, and custom Kubernetes resources with a controller that evaluates the plant against a declared policy, and we ran them through a feed loss and a slow drift. The controller flagged the feed loss 1.15 h of process time after it occurred. In the drift, cost fell 37% in 30 h while the plant moved toward shutdown, and the non-compliant verdict came from a declared level limit, 2 h before the shutdown. From this experience we draw a first draft of requirements for such a layer, which our implementation meets only in part. The work in progress is to refine this draft and check it with a new round of runs.**

***Index Terms*—Kubernetes, supervisory control, process automation, Industry 4.0, OPC UA, Tennessee Eastman Process**

I. Introduction

Process plants are operated in layers [1]. Controllers keep variables such as levels and pressures at their setpoints, and a supervisory layer chooses those setpoints and checks whether the plant does what was decided. In the enterprise-control hierarchy of IEC 62264 [2], this supervision sits between the control systems and the business systems. In practice, the engineers who know how to improve a plant often work on data exported from the historian and analyzed offline.

Software engineering faces a similar problem in distributed systems, and Kubernetes addresses it with a declarative interface: the user declares the desired state, and controllers continuously compare it with the observed state [3]. This suggested to us a supervisory layer in which the operating policy of a plant is a declared resource and a controller evaluates the plant against it. An operating change then becomes declared, versioned and recorded.

We built such a layer, in which a Kubernetes controller follows a simulated plant, reached over OPC UA, against a declared policy. The arrangement worked from end to end, but this did not answer whether the plant was supervised. An information security standard such as ISO/IEC 27001 [4] requires an organization to show that certain processes exist, regardless of the tools it uses. We came to ask the same about our work: what must an implementation demonstrate before one can say that a plant is under Kubernetes-based supervision? This paper reports what we built, what happened when we ran it, and the first draft of requirements that came out of it. We started without a specification, and the draft is a result of the experience, not a standard.

II. Related Work

Kubernetes has reached industrial automation mainly as infrastructure for control software. Johansson et al. use it to recover failed controllers with high availability [5], Mellado and Núñez run a programmable logic controller in containers [6], and Koziolek and Burger use a purpose-built Kubernetes operator to update virtual controllers without halting production [7]. In these cases, Kubernetes manages the software that controls the process, not the decisions about how the process should operate.

Two industrial initiatives are closer in intent. The NAMUR Open Architecture (NOA) adds a separate communication path that makes production data available for monitoring and optimization, with no impact on the automation system [8]. O-PAS defines conformance profiles for interoperable process automation products, verified by independent laboratories [9]. Our layer shares the separation of NOA, and our draft shares with O-PAS the idea of claims against a specification, but it concerns supervisory behavior and, at this stage, asks for evidence rather than a test suite. These works do not say what a Kubernetes-based supervisory layer must demonstrate.

III. Kubernetes in Brief

Kubernetes stores the desired state of a system as objects called *resources*. Programs called *controllers* watch these objects, compare them with what they observe and act on the difference, in a loop called *reconciliation* [3]. Three concepts allow this mechanism to be extended [10]:

```
kind: OperatingPolicy
metadata: {name: tep-mode1}
spec:
  costFunctionRef: tep-downs-vogel
  windowSeconds: 60
  maxCost: 170.6
  persistenceEvaluations: 3
  targets:
    - {signal: xmeas.stream11.flow_rate,
       value: 22.949, tolerancePercent: 5}
  constraints:
    - {signal: xmeas.reactor.level,
       min: 50, max: 100}
  # ... other targets, limits and control loops
```

Fig. 1. Excerpt of a descriptor: the engineer declares the cost budget, targets and limits, and how many bad evaluations in a row change the verdict.

- A *custom resource definition* (CRD) registers a new type of resource in the Kubernetes API, with its name, fields and validation rules. It has no behavior of its own.
- A *custom resource* is one object of that type, usually written as a text file, the *descriptor*.
- A *custom controller* is the program that gives the type its behavior. The combination of CRDs and a controller is called an *operator*.

The standard objects of Kubernetes do not help here: a Pod or a Deployment describes a program and whether it is running, and has no notion of a reactor level, a cost budget or an operating limit. All of them can be healthy while the plant moves toward shutdown, so supervision in Kubernetes requires new resource types and a controller that evaluates them.

IV. What We Built

We wanted a plant that runs continuously and can be reached like real equipment, so we rewrote the Downs and Vogel model of the Tennessee Eastman Process (TEP) [11] as a program that publishes its signals over OPC UA. In steady state, it reproduces the published base-case cost (170.63 against 170.61 \$/h), which checks the model at equilibrium but not its dynamics. All the code is open source[1] and Fig. 2 shows its parts, on a single-node local cluster. A historian keeps the last 15 minutes of signals and serves window statistics and a control loop performance index [12], so Kubernetes never sees raw signals.

We wrote three CRDs without a prior specification, to try the idea. `CostFunction` declares the operating cost as a sum of terms, `OperatingPolicy` declares what the plant should comply with (Fig. 1), and `Plant` links them and holds the verdict. The process engineer writes the descriptor and applies it with `kubectl apply`. Every 30 s, and whenever the policy is edited, the controller queries the historian and writes in the `Plant` status a condition for each criterion and a phase, compliant or not. Reconciliation here ends at the verdict: the controller never writes to the plant.

[1]https://github.com/Green-Cinnamon-Labs.
Start at the `spec-tennessee-eastman` repository.

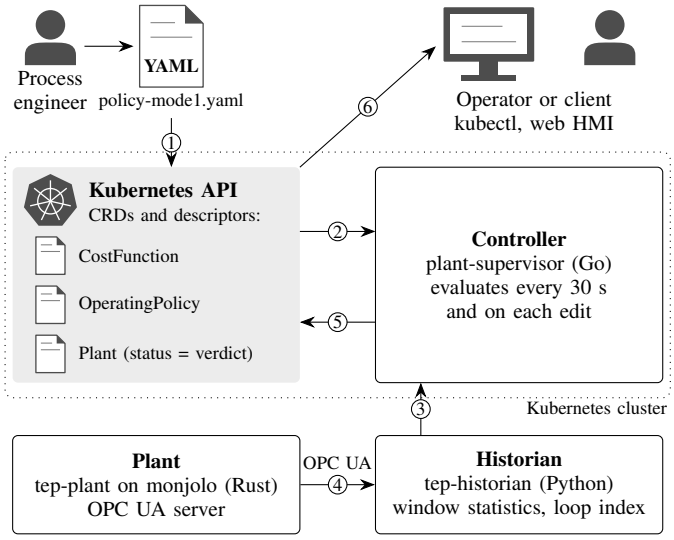


Fig. 2. Architecture of the first implementation. Boxes are the applications we developed. 1: the engineer applies the descriptor. 2: the controller watches the resources. 3: it queries aggregates. 4: the historian reads the plant over OPC UA. 5: the controller writes the verdict. 6: clients read it.

V. What Happened

All times below are process times. The plant was accelerated, but the 30 s evaluation period is wall-clock time, so each evaluation covered more process time than it would in real operation.

Normal operation. We set the cost budget at 170.6 \$/h, the base-case cost, and calibrated the thresholds on a run without disturbance at about ten times real time. Over 41 evaluations, the cost stayed at 166.83 ± 0.18 \$/h and all conditions held.

Feed loss. At the same speed (about 5 min of process per evaluation), we applied disturbance IDV(6), the loss of feed A (Fig. 3). The product flow left its target 0.98 h after the disturbance, and the cost exceeded the budget at 1.15 h (about 7 min of wall-clock time), when the phase changed to non-compliant. The verdict followed the physical order of events. We then added a criterion on the setpoint offset, which required changing the historian, the CRD and the controller, not only the policy. In a repeated run, the levels crossed the declared offset at 0.84 h and the loop condition turned false at 1.00 h.

Cost that improves while the plant degrades. The plant had not recovered after the feed loss, so we changed the three control loops from proportional to PI, a plausible design choice that still left the reactor level without a loop, and ran the plant without disturbance at about 56 times real time (about 28 min of process per evaluation). With proportional loops, the same plant converged. With PI, the purge valve closed slowly and the reactor level fell until the reactor pressure crossed the 3000 kPa shutdown limit at 32.9 h (Fig. 4). The cost fell from 165.9 to 105.3 \$/h in 30 h, because less reactant left in the purge, and the cost condition held in all 70 evaluations. The level crossed the declared 50% limit at 29.1 h. In Kubernetes, the condition `ConstraintsSatisfied` turned false at

TABLE I
FROM OBSERVATIONS TO DRAFT REQUIREMENTS

What we observed	An implementation should demonstrate that	Our implementation
The policy could be edited with the controller running.	The policy is a declared resource, and an edit takes effect within one evaluation.	Built; delay not measured.
Verdicts could be read by any client, with their times.	Each verdict is recorded with its time as Kubernetes state.	Met.
We designed the controller to withhold a verdict without data.	Missing data produce no verdict, and this is stated.	Built; not exercised in the runs.
The controller only reads, and control stays in the plant.	Control and safety stay in the plant controllers, and the plant stays safe if the supervisor fails.	Only reads; supervisor failure not tested.
The feed loss was flagged at 1.15 h.	A reference scenario is detected within a limit declared in advance.	No limit declared yet.
Criteria come from the literature and are coded by each implementation.	Each criterion is checked against its source on a known scenario before it is trusted.	Not done yet.
A new type of criterion required changing code.	It is stated which changes need only a descriptor and which need code.	Thresholds in descriptors; new criteria in code.
Cost improved while the level fell, in a variable with no loop.	The policy covers the variables whose loss stops the operation, including trends.	Limits only; no trends.
The verdict came 2 h before the shutdown.	The verdict leaves a declared margin to act.	No margin declared yet.
What a window covers changes with the plant speed.	Windows and periods state their time base, and judgments do not depend on simulation speed.	Windows in wall-clock time.

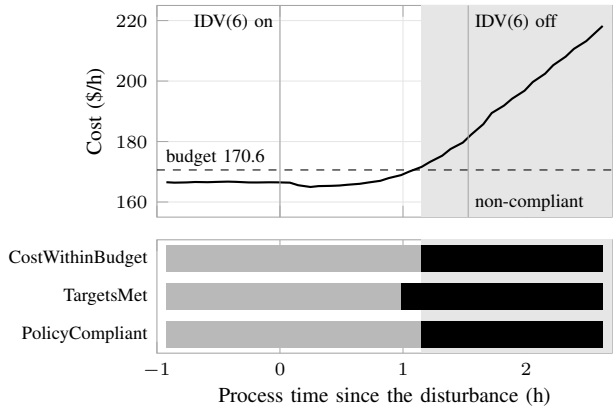


Fig. 3. Feed loss. Cost from the supervisor status and the Kubernetes conditions that changed (gray: true, black: false). The cost first dips, because less feed A leaves in the purge, and keeps rising after the disturbance ends. Shaded: non-compliant phase.

30.0 h, and the phase turned non-compliant at 30.9 h, after the three bad evaluations required by the policy. Part of this delay comes from the acceleration, but the level had also fallen for about 25 h inside the allowed range: a fixed limit warns when a variable arrives, not when it starts to drift.

Predictability index. The bottom panel of Fig. 4 shows the index of Bradu et al. [12]: the more of the error of a control loop an autoregressive model predicts, the lower the index, which suggests that the loop can improve. We computed it offline from the recordings, with the controller's persistence and a threshold of 0.5 calibrated on the normal run. These values are not the loop condition recorded by the controller, which we do not show, and the phase does not depend on them. The index flagged the loops from 2.1 to 12.6 h, mostly the pressure loop, and again at the last evaluation, but not during the fall of the level, a variable with no loop. It also flagged 4% of the evaluations in the converging run, against 33% here. At this speed, its 300 s wall-clock window covers about 4.7 h of process, so we take these values as preliminary.

VI. From the Lessons to a Draft Specification

Table I turns what we observed into draft requirements, and shows where our implementation stands. Like ISO/IEC 27001 [4], which states requirements and is checked by audit, the draft does not fix numerical limits or prescribe tests, because they depend on the plant and on the criteria chosen. The implementation declares its thresholds in advance and presents evidence, and the decision comes from comparing the two. A scenario test is one way to produce evidence.

VII. Ongoing Work

The draft in Table I covers only observation, because this is all we have tried. It is the main work in progress: we will make its requirements precise, adjust our implementation to them, declare the thresholds in advance, and carry out a new round of runs, whose data we intend to present. In that round, the controller will judge the loops with the index on the scale of its source and a recalibrated threshold.

Acting on the plant. When the supervisor only observes, a correct verdict may come too late to act on. We are designing the supervisor to treat a setpoint change as Kubernetes treats a software deployment: applied in steps, verified, tested first on a digital twin, and rolled back if a declared condition fails. We expect this to add requirements to the draft.

Second plant and other groups. The next plant is a didactic level process at Ifes, where a PID in the plant controller drives a pump and the setpoint is exposed through an OPC server. Its model will first be written in `monjolo`; experiments on the physical plant depend on access and authorization, which have not been confirmed yet. We also plan to simplify the stack, with a Python interface for plant models, a historian template and a single-command cluster setup. The target is concrete: a second group brings its own plant under supervision with

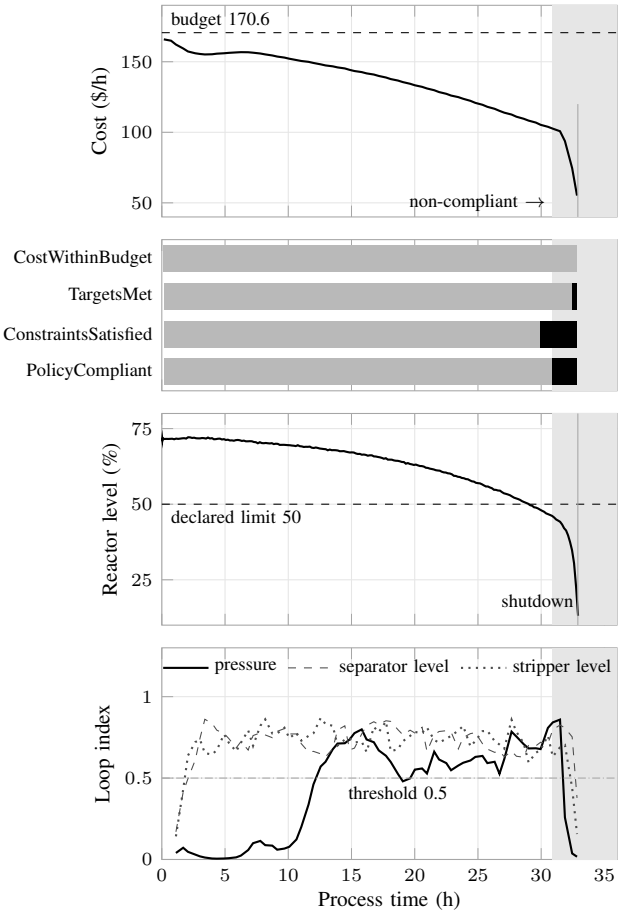


Fig. 4. Run with PI loops. From the top: cost from the supervisor status; Kubernetes conditions (gray: true, black: false); reactor level from the historian; predictability index on the scale of its source, computed offline from the recordings. Shaded: non-compliant phase.

no changes to the controller code, only new descriptors, and presents its evidence.

VIII. Conclusion

We asked what an implementation must demonstrate before a plant can be said to be under Kubernetes-based supervision, and we approached the question by building one. We learned four things. The arrangement works from end to end: a Kubernetes controller followed a simulated plant, reached over OPC UA, against a declared policy, and within Kubernetes this requires CRDs and a controller, because the standard objects know nothing of the process. A detection delay can only be judged against a limit declared in advance. A policy must watch more than cost, which can improve while the plant degrades. And observation alone may leave little time to act.

These lessons became a first draft of requirements, which our implementation meets only in part. The work that remains is to make the draft precise and check it with a new round of runs, to test the layer in real time, on a physical plant and on a multi-node cluster, to let the supervisor act on the plant, and to have a second group bring its own plant under supervision.

Acknowledgment

The text of all sections and the code of Figs. 2 to 4 were drafted with Claude Opus 5.5 (Anthropic), through Claude Code, from the authors' thesis, software and notes. The authors reviewed the content and are responsible for it.

References

[1] S. Skogestad, "Control structure design for complete chemical plants," *Computers & Chemical Engineering*, vol. 28, no. 1–2, pp. 219–234, 2004.

[2] *Enterprise-Control System Integration – Part 1: Models and Terminology*, IEC 62264-1:2013, 2013.

[3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.

[4] *Information Security, Cybersecurity and Privacy Protection – Information Security Management Systems – Requirements*, ISO/IEC 27001:2022, 2022.

[5] B. Johansson, M. Rågberger, T. Nolte, and A. V. Papadopoulos, "Kubernetes orchestration of high availability distributed control systems," in *2022 IEEE International Conference on Industrial Technology (ICIT)*, 2022, pp. 1–8.

[6] J. Mellado and F. Núñez, "Design of an IoT-PLC: a containerized programmable logical controller for the Industry 4.0," *Journal of Industrial Information Integration*, vol. 25, p. 100250, 2022.

[7] H. Koziolek and A. Burger, "Dynamic updates of virtual PLCs deployed as Kubernetes microservices," in *Software Engineering 2022*, ser. Lecture Notes in Informatics, vol. P-320. Gesellschaft für Informatik, 2022.

[8] *NAMUR Open Architecture – NOA Concept*, NAMUR Recommendation NE 175, 2021.

[9] *O-PAS Standard, 2nd Edition*, The Open Group Standard. [Online]. Available: https://publications.opengroup.org/standards/opa

[10] The Kubernetes Authors, "Operator pattern," Kubernetes documentation, accessed Nov. 3, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/extend-kubernetes/operator/

[11] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–255, 1993.

[12] B. Bradu, E. Blanco Viñuela, R. Marti, and F. Tilaro, "Automatic PID performance monitoring applied to LHC cryogenics," in *16th International Conference on Accelerator and Large Experimental Physics Control Systems (ICALEPCS 2017)*, 2018, p. WEAPL02.